

Enhancing GLM-Based Lapse Rate Models Using MID - EV Valuation Tutorial

Reiji Kozuma^{*} Ryoichi Asashiba[†] Yoshihiro Matsumori[‡] Hirokazu Iwasawa[§]

2026-09-25

Abstract

This document provides supplementary code and documentation for the Embedded Value (EV) calculations conducted in the original research paper ([Kozuma et al., 2026](#)). The theoretical concepts and valuation frameworks introduced in the paper are assumed as background knowledge.

Table of Contents

1 Preparation

- 1.1 Preparing Packages and Data
- 1.2 Building XGBoost Models

2 Implementing EV Calculation

- 2.1 Disaggregating Policy Cohorts
- 2.2 Helper Functions for In-Force Portfolio Roll-Forward
- 2.3 Projecting Future Gross Premium Cash Flows
- 2.4 Cash Flow Projection and Present Value Calculation

3 Calculating EV Analysis of Movement

Acknowledgments

References

^{*} Daido Life Insurance Company, rktkdm@gmail.com

[†] Daido Life Insurance Company, ryoichi.asashiba@gmail.com

[‡] Sompo Japan Insurance Inc., ymatsumori1@sompo-japan.co.jp

[§] Graduate School of Accountancy, Waseda University, iwahiro@bb.mbn.or.jp

1 Preparation

1.1 Preparing Packages and Data

Load the R packages required for EV modeling and visualization.

```
library(midr)
library(tidyverse)
library(readxl)
library(xgboost)
library(waterfalls)
```

Data loading and basic preprocessing are identical to the basic tutorial and are therefore omitted here. We define helper functions to extract required predictor and target subsets from the dataset.

```
# Convert to model matrix
get_modelmatrix <- function(df){
  df <- prep_toint_pjratio(df)
  df <- model.matrix(~ ., df) # An intercept column of all 1s is included
}

get_xo <- function(df){ # All predictors and target variables
  df %>% select(setdiff(names(df),
                        c("ID", "lapse_study_year", "policy_year", "issue_year",
                          "exposure_amount")))
}

get_x <- function(df){ # Predictors only
  df <- df %>% get_xo
  df <- df %>% select(setdiff(names(df), c("lapse", "exposure", "lapse_rate")))
}
```

1.2 Building XGBoost Models

We train XGBoost predictive models following the same procedure and hyperparameter settings as in the basic tutorial. Because this tutorial focuses on the EV Analysis of Movement (AoM) from year-end 2002 to year-end 2003 (evaluation year 2002), models corresponding to both opening and closing dates must be estimated.

```
year_start <- 2002
year_end <- 2003
df_train_start <- df_all %>%
  filter(lapse_study_year >= year_start-2, lapse_study_year <= year_start)
df_test_start <- df_all %>%
  filter(lapse_study_year == year_start+1)
df_train_end <- df_all %>%
  filter(lapse_study_year >= year_end-2, lapse_study_year <= year_end)
df_test_end <- df_all %>%
  filter(lapse_study_year == year_end+1)
```

```

get_mm <- function(df){
  df <- df %>%
    select(setdiff(names(df),c(
      "ID", "lapse_study_year", "policy_year", "issue_year",
      "exposure_amount", "lapse", "exposure", "lapse_rate")) %>%
      prep_toint_pjratio
    model.matrix(~ ., df)
}
df_train_start_mm <- df_train_start %>% get_mm
constant_cols_start <- which(apply(df_train_start_mm, 2, function(col) all(col == col[1])))
dm_train_start <- xgb.DMatrix(
  data = df_train_start_mm[, -constant_cols_start, drop = FALSE],
  label = df_train_start %>% pull(lapse_rate),
  weight = df_train_start %>% pull(exposure))
df_train_end_mm <- df_train_end %>% get_mm
constant_cols_end <- which(apply(df_train_end_mm, 2, function(col) all(col == col[1])))
dm_train_end <- xgb.DMatrix(
  data = df_train_end_mm[, -constant_cols_end, drop = FALSE],
  label = df_train_end %>% pull(lapse_rate),
  weight = df_train_end %>% pull(exposure))

```

```

set.seed(2025)
cv_xgboost_start <- xgb.cv(data = dm_train_start, nrounds = 2000, nfold = 4,
  early_stopping_rounds = 10, params = params_xgb, verbose = 0)
nrounds <- cv_xgboost_start$early_stop$best_iteration

set.seed(2025)
model_xgboost_start_raw <- xgb.train(data = dm_train_start, nrounds = nrounds,
  params = params_xgb)

set.seed(2025)
cv_xgboost_end <- xgb.cv(data = dm_train_end, nrounds = 2000, nfold = 4,
  early_stopping_rounds = 10, params = params_xgb, verbose = 0)
nrounds <- cv_xgboost_end$early_stop$best_iteration

set.seed(2025)
model_xgboost_end_raw <- xgb.train(data = dm_train_end, nrounds = nrounds,
  params = params_xgb)

```

To streamline downstream projection logic, we package prediction execution into a unified function. If certain features were constant and dropped during training, they must be similarly removed when scoring new datasets. We bundle this handling alongside the model object in a dedicated list.

```

model_xgboost_start <- list(
  model = model_xgboost_start_raw,
  constant_cols = constant_cols_start
)
model_xgboost_end <- list(

```

```

model = model_xgboost_end_raw,
constant_cols = constant_cols_end
)

predfun_xgboost <- function(model, newdata){
  tmp <- newdata %>% get_x %>% get_modelmatrix
  tmp <- tmp[, -model$constant_cols, drop = FALSE]
  predict(model$model, tmp)
}

```

2 Implementing EV Calculation

2.1 Disaggregating Policy Cohorts

In cash flow projections, policy duration is incremented year by year to project future lapse rates. However, because duration is aggregated into broad bands in the study dataset, these cohorts must be disaggregated into single policy-year records.

```

df_all %>% group_by(lapse_study_year, duration, issue_year) %>%
  summarise(sum(exposure), .groups = "drop")

```

A tibble: 91 x 4

	lapse_study_year	duration	issue_year	`sum(exposure)`
	<int>	<ord>	<ord>	<dbl>
1	2000	06-09	1991-1993	36406.
2	2000	06-09	1994-1996	60465.
3	2000	10	1991-1993	14055.
4	2000	11	-1990	3746.
5	2000	12	-1990	348.
6	2000	13-	-1990	185
7	2001	06-09	1991-1993	23212.
8	2001	06-09	1994-1996	113966.
9	2001	10	1991-1993	18372.
10	2001	11	1991-1993	10989.

i 81 more rows

By leveraging `issue_year` information, the feasible range of `duration` can be narrowed down.

For instance, for records with `lapse_study_year == 2000`, `duration == 6-9`, and `issue_year == 1991-1993`, the relationship `duration = lapse_study_year - issue_year + 1` mathematically yields possible durations of 8 through 10. However, duration 10 contradicts the condition `duration == 6-9`. Thus, the remaining possibilities are durations 8 and 9.

Because no further granular information is available, we split the record equally into two records representing durations 8 and 9, respectively. Similarly, whenever multiple durations are possible for a given key combination, exposures are allocated equally across the valid durations.

Additionally, for the aggregated cohort `issue_year == "Pre-1991"`, inspecting records at

`lapse_study_year == 2000` reveals that exposures at `duration == 13-` (policies issued in 1988 or earlier) are negligible compared to other duration bands.

```
df_all %>% filter(issue_year == "-1990", lapse_study_year < 2004) %>%
  group_by(lapse_study_year, duration, issue_year) %>%
  summarise(sum(exposure), .groups = "drop")
```

A tibble: 7 x 4

	lapse_study_year	duration	issue_year	`sum(exposure)`
	<int>	<ord>	<ord>	<dbl>
1	2000	11	-1990	3746.
2	2000	12	-1990	348.
3	2000	13-	-1990	185
4	2001	12	-1990	2766.
5	2001	13-	-1990	435.
6	2002	13-	-1990	2830.
7	2003	13-	-1990	2590.

Assuming virtually zero policy volume for contracts issued in 1987 or earlier, we treat Pre-1991 policies as having been issued in 1988 or later.

We denote the disaggregated duration as `duration_estimated`. The corresponding issue year derived from `issue_year = lapse_study_year - duration_estimated + 1` is similarly designated as `issue_year_estimated`.

In practice, this disaggregation is performed by pre-building a mapping table that links original cohort keys to valid `duration_estimated` values and split proportions, and merging it via `left_join()`.

```
df_conv_issue_year_estimated <- data.frame(
  issue_year = c("-1990", "-1990", "-1990",
                 "1991-1993", "1991-1993", "1991-1993",
                 "1994-1996", "1994-1996", "1994-1996",
                 "1997-1999", "1997-1999", "1997-1999",
                 "2000-2002", "2000-2002", "2000-2002",
                 "2003-2004", "2003-2004",
                 "2005-2007", "2005-2007", "2005-2007") %>%
  factor(levels = levels(df_all$issue_year), ordered = TRUE),
  issue_year_estimated = 1988:2007
)

df_conv_duration_estimated <- data.frame(
  duration = c("06-09", "06-09", "06-09", "06-09",
              "10", "11", "12",
              rep("13-", 55 - 13 + 1)) %>%
  factor(levels = levels(df_all$duration), ordered = TRUE),
  duration_estimated = 6:55
)
```

```

# Enumerate all valid combinations
df_conv_estimated <- expand.grid(
  lapse_study_year = unique(df_all$lapse_study_year),
  duration = unique(df_all$duration),
  issue_year = unique(df_all$issue_year)
) %>%
left_join(df_conv_issue_year_estimated, join_by(issue_year == issue_year), relationship =
  ↪ "many-to-many") %>%
left_join(df_conv_duration_estimated, join_by(duration == duration), relationship =
  ↪ "many-to-many") %>%
filter(issue_year_estimated + duration_estimated - 1 == lapse_study_year)

# Calculate split proportions for disaggregating records
df_conv_estimated <- df_conv_estimated %>% left_join(
  df_conv_estimated %>% group_by(lapse_study_year, duration, issue_year) %>%
    summarise(exposure_rate = 1/n(), .groups = "drop"),
  join_by(lapse_study_year == lapse_study_year,
    duration == duration,
    issue_year == issue_year)
) %>% arrange(lapse_study_year, duration, issue_year)

# Function to disaggregate records into single-year durations
fn_sep_to_estimated <- function(df_tmp){
  df_tmp <- df_tmp %>% left_join(df_conv_estimated,
    join_by(lapse_study_year == lapse_study_year,
      duration == duration,
      issue_year == issue_year), relationship = "many-to-many")
  for(col in c("exposure_amount", "exposure_count", "lapse_amount", "lapse_count",
    ↪ "exposure", "lapse")){
    if(col %in% names(df_tmp)){
      df_tmp <- df_tmp %>% mutate(!col := .data[[col]] * exposure_rate)
    }
  }
  df_tmp %>% select(-exposure_rate)
}

```

For example, consider the following raw records:

```

df_tmp <- df_train_start[1:5, ]
df_tmp %>%
  select(ID, lapse_study_year, policy_year, duration,
    exposure, lapse, exposure_amount)

```

A tibble: 5 x 7

	ID	lapse_study_year	policy_year	duration	exposure	lapse	exposure_amount
	<int>	<int>	<int>	<ord>	<dbl>	<dbl>	<dbl>
1	1	2000	2000	06-09	7	1	325000
2	3	2000	2000	06-09	7	0	2600000
3	13	2001	2001	10	5	5	1800000

4	20	2000	2000 06-09	4	0	500000
5	25	2002	2002 06-09	32	3	3880000

Applying the disaggregation function yields the following expanded records:

```
df_tmp %>%
  fn_sep_to_estimated %>%
  select(ID, lapse_study_year, policy_year, duration, issue_year_estimated,
         duration_estimated, exposure, lapse, exposure_amount)
```

A tibble: 10 x 9

	ID	lapse_study_year	policy_year	duration	issue_year_estimated
	<int>	<int>	<int>	<ord>	<int>
1	1	2000	2000	06-09	1994
2	1	2000	2000	06-09	1995
3	3	2000	2000	06-09	1994
4	3	2000	2000	06-09	1995
5	13	2001	2001	10	1992
6	20	2000	2000	06-09	1992
7	20	2000	2000	06-09	1993
8	25	2002	2002	06-09	1994
9	25	2002	2002	06-09	1995
10	25	2002	2002	06-09	1996

	duration_estimated	exposure	lapse	exposure_amount
	<int>	<dbl>	<dbl>	<dbl>
1	7	3.5	0.5	162500
2	6	3.5	0.5	162500
3	7	3.5	0	1300000
4	6	3.5	0	1300000
5	10	5	5	1800000
6	9	2	0	250000
7	8	2	0	250000
8	9	10.667	1	1293333.
9	8	10.667	1	1293333.
10	7	10.667	1	1293333.

2.2 Helper Functions for In-Force Portfolio Roll-Forward

We define several helper functions to roll the in-force portfolio forward. The `fn_shift_year()` function advances policy duration by adding a specified number of years to `policy_year` and `duration_estimated`.

```
fn_shift_year <- function(df_sep, year){
  df_sep %>% mutate(policy_year = policy_year + year) %>%
```

```
mutate(duration_estimated = duration_estimated + year)
}
```

For example, advancing the previous sample records by 5 years produces the following table:

```
df_tmp %>%
  fn_sep_to_estimated %>%
  fn_shift_year(year = 5) %>%
  select(ID, lapse_study_year, policy_year, duration, issue_year_estimated,
         duration_estimated, exposure, lapse, exposure_amount)
```

A tibble: 10 x 9

	ID	lapse_study_year	policy_year	duration	issue_year_estimated
	<int>	<int>	<dbl>	<ord>	<int>
1	1	2000	2005	06-09	1994
2	1	2000	2005	06-09	1995
3	3	2000	2005	06-09	1994
4	3	2000	2005	06-09	1995
5	13	2001	2006	10	1992
6	20	2000	2005	06-09	1992
7	20	2000	2005	06-09	1993
8	25	2002	2007	06-09	1994
9	25	2002	2007	06-09	1995
10	25	2002	2007	06-09	1996

	duration_estimated	exposure	lapse	exposure_amount
	<dbl>	<dbl>	<dbl>	<dbl>
1	12	3.5	0.5	162500
2	11	3.5	0.5	162500
3	12	3.5	0	1300000
4	11	3.5	0	1300000
5	15	5	5	1800000
6	14	2	0	250000
7	13	2	0	250000
8	14	10.667	1	1293333.
9	13	10.667	1	1293333.
10	12	10.667	1	1293333.

The `fn_get_port_eopy()` function decrements the portfolio by actual observed lapses to construct the actual end-of-year in-force portfolio.

```
fn_get_port_eopy <- function(df_sep){
  # If only exposure_amount is present, decrement it proportionally using exposure counts
  if ("exposure" %in% names(df_sep) & "lapse" %in% names(df_sep) &
      "exposure_amount" %in% names(df_sep) & !("lapse_amount" %in% names(df_sep))){
```

```

df_sep <- df_sep %>%
  mutate(exposure_amount = exposure_amount * (exposure - lapse) / exposure)
}
for(col in list(c("exposure_amount", "lapse_amount"),
  c("exposure_count", "lapse_count"),
  c("exposure", "lapse"))){
  if(col[1] %in% names(df_sep) & col[2] %in% names(df_sep)){
    df_sep <- df_sep %>%
      mutate(!col[1] := .data[[col[1]]] - .data[[col[2]]]) %>%
      mutate(!col[2] := 0)
  }
}
df_sep %>% fn_shift_year(year = 1)
}

```

Applying this decrement function to our sample records yields:

```

df_tmp %>%
  fn_sep_to_estimated %>%
  fn_get_port_eopy %>%
  select(ID, lapse_study_year, policy_year, duration, issue_year_estimated,
    duration_estimated, exposure, lapse, exposure_amount)

```

A tibble: 10 x 9

	ID	lapse_study_year	policy_year	duration	issue_year_estimated
	<int>	<int>	<dbl>	<ord>	<int>
1	1	2000	2001	06-09	1994
2	1	2000	2001	06-09	1995
3	3	2000	2001	06-09	1994
4	3	2000	2001	06-09	1995
5	13	2001	2002	10	1992
6	20	2000	2001	06-09	1992
7	20	2000	2001	06-09	1993
8	25	2002	2003	06-09	1994
9	25	2002	2003	06-09	1995
10	25	2002	2003	06-09	1996

	duration_estimated	exposure	lapse	exposure_amount
	<dbl>	<dbl>	<dbl>	<dbl>
1	8	3	0	139286.
2	7	3	0	139286.
3	8	3.5	0	1300000
4	7	3.5	0	1300000
5	11	0	0	0
6	10	2	0	250000
7	9	2	0	250000

8	10	9.6667	0	1172083.
9	9	9.6667	0	1172083.
10	8	9.6667	0	1172083.

When scoring models, the disaggregated *_estimated variables must be mapped back to the original categorical factors expected by the prediction pipeline. The `fn_set_from_estimated()` function handles this conversion.

```
fn_set_from_estimated <- function(df_sep){
  df_sep %>%
    left_join(
      df_conv_duration_estimated %>% rename(duration_new = duration),
      join_by(duration_estimated == duration_estimated),
      relationship = "many-to-one"
    ) %>%
    mutate(duration = duration_new) %>%
    left_join(
      df_conv_issue_year_estimated %>% rename(issue_year_new = issue_year),
      join_by(issue_year_estimated == issue_year_estimated),
      relationship = "many-to-one"
    ) %>%
    mutate(issue_year = issue_year_new) %>%
    select(-duration_estimated, -issue_year_estimated,
          -duration_new, -issue_year_new)
}
```

Converting the shifted records back to original model factors yields:

```
df_tmp %>%
  fn_sep_to_estimated %>%
  fn_shift_year(year = 5) %>%
  fn_set_from_estimated %>%
  select(ID, lapse_study_year, policy_year, issue_year, duration, exposure, lapse)
```

A tibble: 10 x 7

	ID	lapse_study_year	policy_year	issue_year	duration	exposure	lapse
	<int>	<int>	<dbl>	<ord>	<ord>	<dbl>	<dbl>
1	1	2000	2005	1994-1996	12	3.5	0.5
2	1	2000	2005	1994-1996	11	3.5	0.5
3	3	2000	2005	1994-1996	12	3.5	0
4	3	2000	2005	1994-1996	11	3.5	0
5	13	2001	2006	1991-1993	13-	5	5
6	20	2000	2005	1991-1993	13-	2	0
7	20	2000	2005	1991-1993	13-	2	0
8	25	2002	2007	1994-1996	13-	10.7	1
9	25	2002	2007	1994-1996	13-	10.7	1
10	25	2002	2007	1994-1996	12	10.7	1

2.3 Projecting Future Gross Premium Cash Flows

As specified in the paper, future statutory profits are modeled by multiplying projected gross premium cash flows by a constant profit margin ratio. While face amount exposure is recorded in the dataset, gross premium figures are not provided. For term life business, gross premiums can be approximated by multiplying valuation mortality rates by face amount exposure ($q \times S$).

Referencing Page 7 of the SOA report, post-level ART premiums in the US are commonly priced as a percentage of the 2001 CSO mortality table. Accordingly, we adopt the 2001 CSO tables as the pricing benchmark.

The 2001 CSO tables follow a select and ultimate structure categorized by gender, smoking status, and underwriting class. Most tables are sourced from the [2001 CSO Preferred Class Structure Mortality Report & Tables](#), while missing composite categories (e.g., aggregate and undifferentiated smoker) are supplemented from the [SOA Mortality and Other Rate Tables](#) repository.

Both Age Nearest Birthday (ANB) and Age Last Birthday (ALB) tables exist; we assume ANB, which is standard in US life insurance practice.

```
df_mort_dur6 <- read_excel("cso\\2001 CSO.xlsx", sheet = "Dur6", range = "A1:D1801")
df_mort_ult <- read_excel("cso\\2001 CSO.xlsx", sheet = "Ult", range = "A1:D2179")

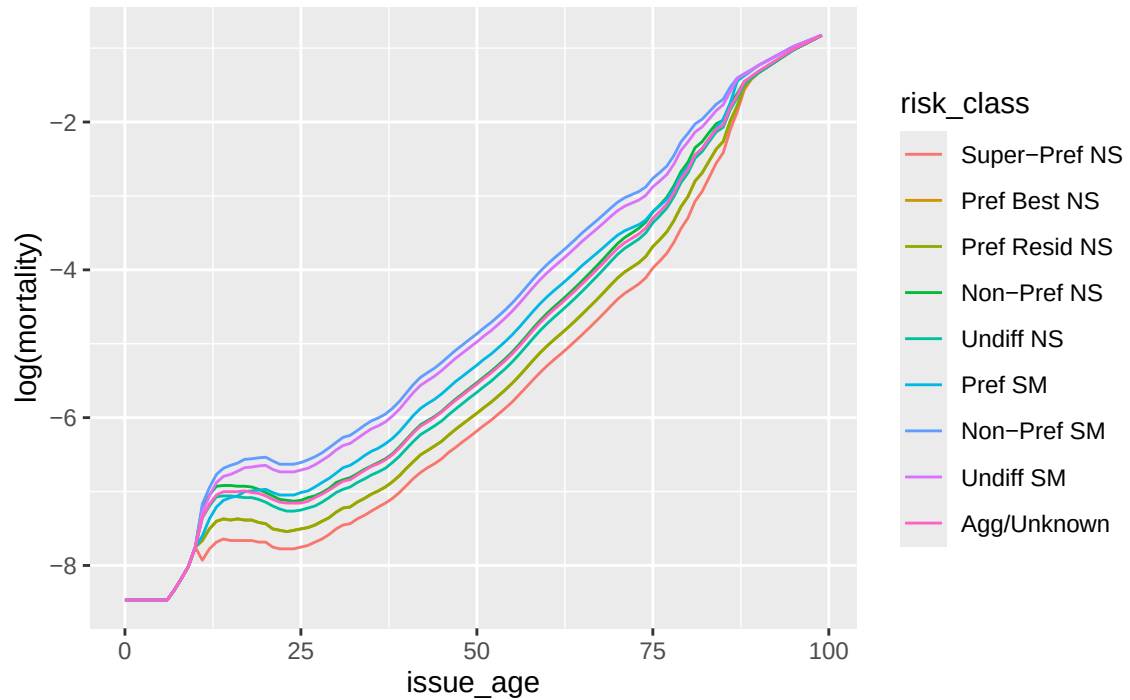
df_mort_dur6 <- df_mort_dur6 %>%
  mutate(risk_class = factor(risk_class, levels = levels(df_all$risk_class))) %>%
  mutate(gender = factor(gender, levels = c("M", "F"),
    labels = levels(df_all$gender)))
df_mort_ult <- df_mort_ult %>%
  mutate(risk_class = factor(risk_class, levels = levels(df_all$risk_class))) %>%
  mutate(gender = factor(gender, levels = c("M", "F"),
    labels = levels(df_all$gender)))
```

The mapping between policy risk classes and specific 2001 CSO mortality tables follows the definitions in the original paper.

Plotting mortality rates on a logarithmic scale illustrates the rate curves across underwriting classes.

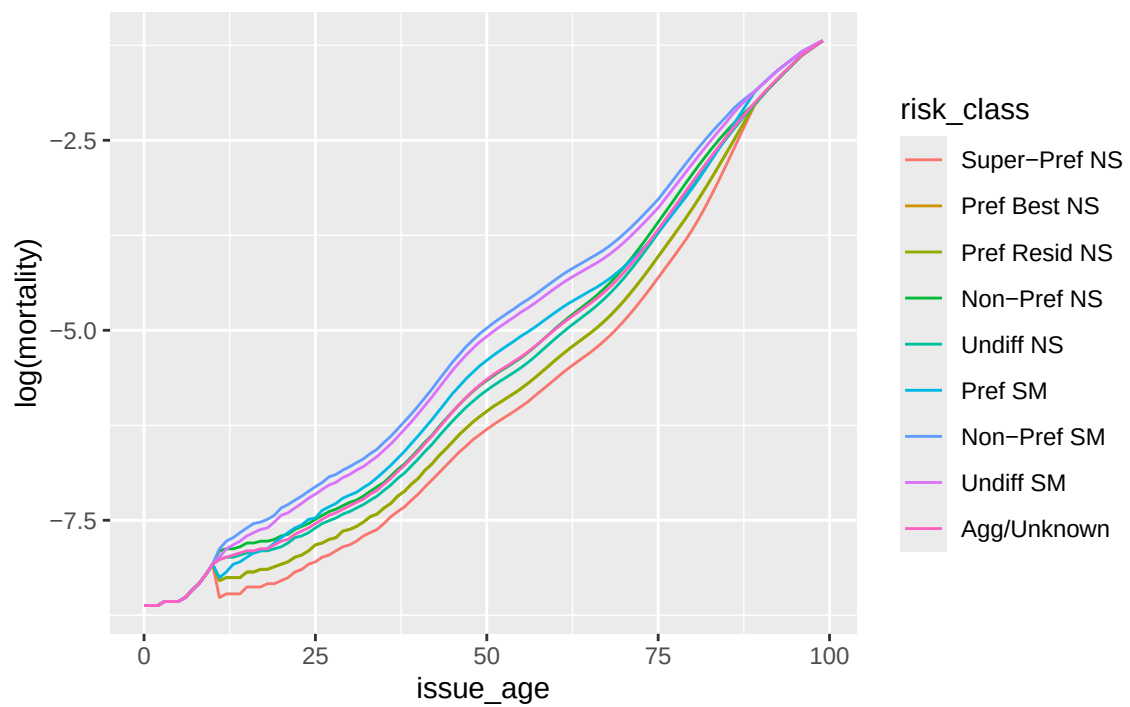
```
ggplot(df_mort_dur6 %>% filter(gender == "male"),
  mapping = aes(x = issue_age, y = log(mortality), group = risk_class, color =
    ↪ risk_class)) +
  geom_line() + ggtitle("2001 CSO Mortality Male Duration = 6")
```

2001 CSO Mortality Male Duration = 6

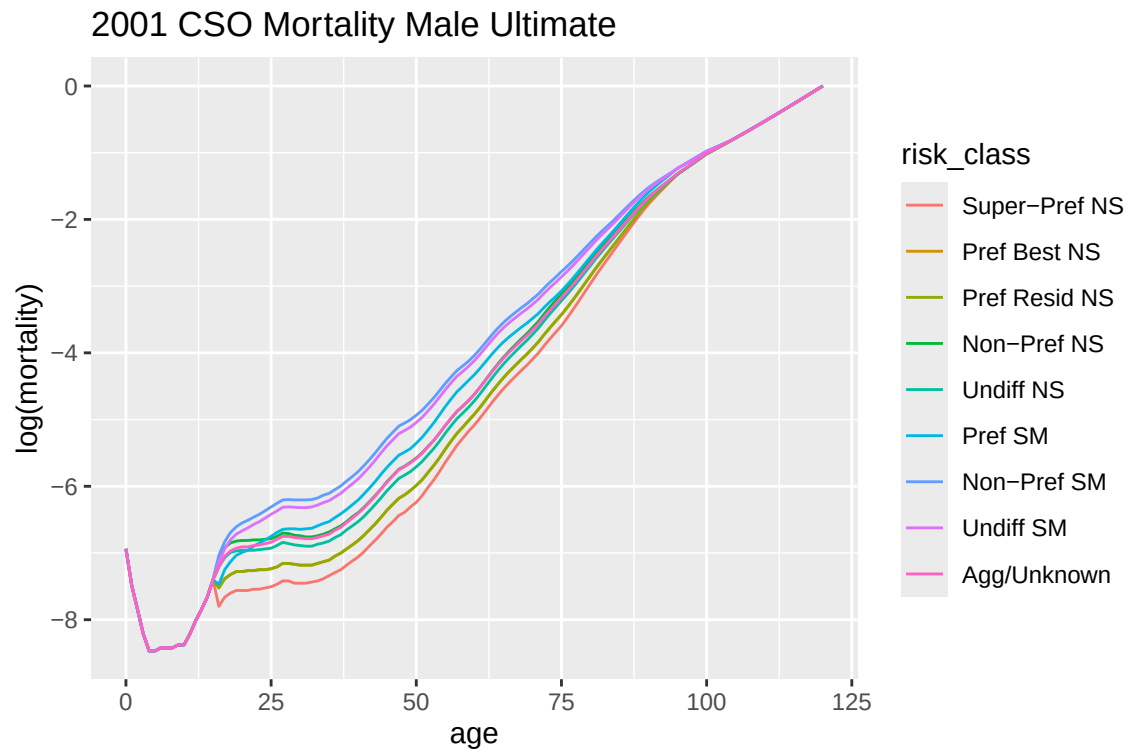


```
ggplot(df_mort_dur6 %>% filter(gender == "female"),
       mapping = aes(x = issue_age, y = log(mortality), group = risk_class, color =
         ⇐ risk_class)) +
  geom_line() + ggtitle("2001 CSO Mortality Female Duration = 6")
```

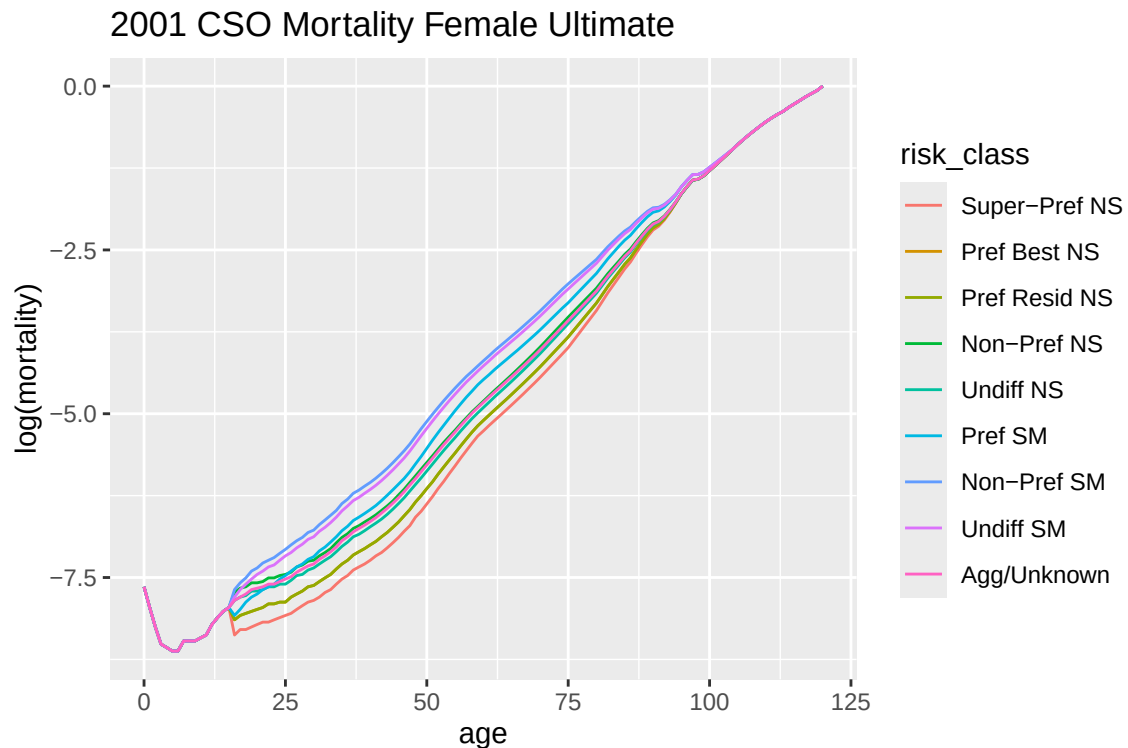
2001 CSO Mortality Female Duration = 6



```
ggplot(df_mort_ult %>% filter(gender == "male"),
       mapping = aes(x = age, y = log(mortality), group = risk_class, color = risk_class))
  ↪ +
  geom_line() + ggtitle("2001 CSO Mortality Male Ultimate")
```



```
ggplot(df_mort_ult %>% filter(gender == "female"),
       mapping = aes(x = age, y = log(mortality), group = risk_class, color = risk_class))
  ↪ +
  geom_line() + ggtitle("2001 CSO Mortality Female Ultimate")
```



Comparing mortality levels shows that undifferentiated classes align closely with their respective residual standard classes. Aggregate non-smoker rates track standard non-smoker rates, while preferred smoker mortality substantially exceeds non-smoker categories, illustrating the pronounced impact of smoking.

The `fn_calc_premium()` function computes gross premium income by multiplying mortality rates from these tables by face amount exposure.

```
fn_calc_premium <- function(df_sep){
  df_sep %>%
    prep_toint_age() %>%
    mutate(issue_age = round(issue_age)) %>%
    mutate(age = issue_age + duration_estimated - 1) %>%
    # Level period uses Select mortality at duration 6 (joined by issue age)
    left_join(
      df_mort_dur6 %>% rename(mortality_dur6 = mortality),
      join_by(issue_age == issue_age, gender == gender, risk_class == risk_class),
      relationship = "many-to-one"
    ) %>%
    # Post-level ART period uses Ultimate mortality (varies by attained age)
    left_join(
      df_mort_ult %>% rename(mortality_ult = mortality),
      join_by(age == age, gender == gender, risk_class == risk_class),
      relationship = "many-to-one"
    ) %>%
    mutate(premium = if_else(duration_estimated <= 10, mortality_dur6, mortality_ult) *
      exposure_amount) %>%

```

```

    pull(premium)
  }

```

Plotting projected premium cash flows for a sample policy demonstrates the sharp increase in premiums at duration 11. While empirical premium jump ratios in the data are somewhat lower than theoretical ART rates, and non-ART policies are included, we apply the standard ART formula across all post-level cohorts for modeling consistency.

```

# Select a sample policy with a relatively large premium_jump_ratio
df_tmp <- df_test_start %>% fn_sep_to_estimated %>%
  filter(duration_estimated == 6, premium_jump_ratio == "10.01-11.00") %>%
  slice_head(n = 1)

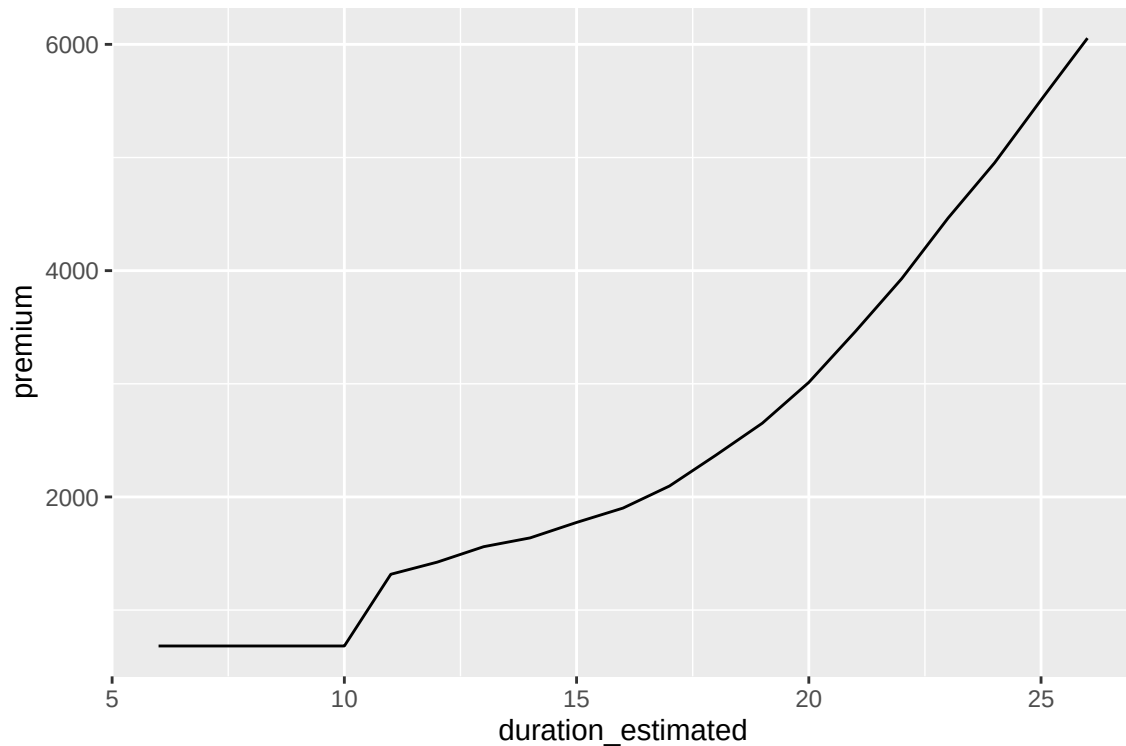
df_tmp_all <- tibble()
for(y in 0:20){
  df_tmp_all <- rbind(df_tmp_all, df_tmp %>% fn_shift_year(year = y))
}
premium_tmp <- df_tmp_all %>% fn_calc_premium

cbind(df_tmp_all, premium = premium_tmp) %>%
  select(ID, policy_year, issue_age, duration_estimated, premium)

```

	ID	policy_year	issue_age	duration_estimated	premium
1	3066	2003	30-39	6	682.50
2	3066	2004	30-39	7	682.50
3	3066	2005	30-39	8	682.50
4	3066	2006	30-39	9	682.50
5	3066	2007	30-39	10	682.50
6	3066	2008	30-39	11	1316.25
7	3066	2009	30-39	12	1423.50
8	3066	2010	30-39	13	1560.00
9	3066	2011	30-39	14	1638.00
10	3066	2012	30-39	15	1774.50
11	3066	2013	30-39	16	1901.25
12	3066	2014	30-39	17	2096.25
13	3066	2015	30-39	18	2369.25
14	3066	2016	30-39	19	2652.00
15	3066	2017	30-39	20	3012.75
16	3066	2018	30-39	21	3461.25
17	3066	2019	30-39	22	3929.25
18	3066	2020	30-39	23	4465.50
19	3066	2021	30-39	24	4953.00
20	3066	2022	30-39	25	5508.75
21	3066	2023	30-39	26	6054.75

```
ggplot(cbind(df_tmp_all, premium = premium_tmp),
       mapping = aes(x = duration_estimated, y = premium)) +
  geom_line()
```



2.4 Cash Flow Projection and Present Value Calculation

We construct a function to calculate the present value of future profits (PVFP). For computational efficiency, discounting is implemented recursively by rolling backward from the final projection period ($L = 20$).

```
# projection_term : Projection Horizon
# verbose :
#   - If >= 1, returns projected cash flows in addition to EV
#   - If 1, returns data for t = 0 only; if 2, returns all projection periods
# int_rate : Valuation Interest Rate
# pm_rate : Profit Margin Ratio (After Tax)
fn_calc_ev <- function(df_sep, model, predfun,
                      projection_term = 20, verbose = 0, int_rate = 0.02,
                      pm_rate = 0.05){

  if (verbose >= 1){
    df_projected_data <- tibble()
  }
  for (t in (projection_term - 1):0){

    df_sep_t <- df_sep %>% fn_shift_year(year = t)
```

```

if(t == projection_term - 1){
  ev <- rep(0, nrow(df_sep))
  if (verbose >= 1){
    lapse_rate <- ev
  }
}else{
  lapse_rate <- predfun(model, df_sep_t %>% fn_set_from_estimated)
  ev <- (1 - lapse_rate) / (1 + int_rate) * ev
  if (verbose >= 1){
    df_projeted_data$cf <- df_projeted_data$cf * (1 - lapse_rate)
  }
}
premium <- df_sep_t %>% fn_calc_premium
if (verbose >= 1){
  df_projeted_data <- rbind(
    data.frame(
      rowid = 1:nrow(df_sep),
      t = rep(t, nrow(df_sep)),
      premium = premium,
      lapse_rate = lapse_rate,
      cf = premium
    ),
    df_projeted_data
  )
}

ev <- ev + premium * pm_rate
}
if (verbose >= 1){
  if (verbose <= 1){
    df_projeted_data <- df_projeted_data %>% filter(t == 0)
  }
  return(list(ev = ev, df_projeted_data = df_projeted_data))
}else{
  return(list(ev = ev))
}
}

```

Evaluating the backward projection on our sample policy produces the following annual cash flow profile. Reflecting the shock lapse at duration 10, surviving premium volumes diminish sharply in subsequent durations.

```

ret <- fn_calc_ev(
  df_tmp,
  model_xgboost_start, predfun_xgboost,
  verbose = 2)
ret$df_projeted_data

```

	rowid	t	premium	lapse_rate	cf
1	1	0	682.50	0.07321494	682.50000

```

119      1  1  682.50 0.07321494 632.53080
118      1  2  682.50 0.07321494 586.22010
117      1  3  682.50 0.07321494 543.30003
116      1  4  682.50 0.72974104 503.52235
115      1  5 1316.25 0.46208256 262.44275
114      1  6 1423.50 0.23819400 152.67548
113      1  7 1560.00 0.16113362 127.46202
112      1  8 1638.00 0.16113362 112.26979
111      1  9 1774.50 0.16113362 102.02763
110      1 10 1901.25 0.16113362  91.70094
19       1 11 2096.25 0.16113362  84.81457
18       1 12 2369.25 0.16113362  80.41389
17       1 13 2652.00 0.16113362  75.50687
16       1 14 3012.75 0.16113362  71.95630
15       1 15 3461.25 0.16113362  69.34761
14       1 16 3929.25 0.16113362  66.03908
13       1 17 4465.50 0.16113362  62.95848
12       1 18 4953.00 0.16113362  58.57944
11       1 19 5508.75 0.00000000  54.65410

```

Multiplying these surviving premium cash flows by the profit margin ratio and discounting at 2.0% yields the EV (PVFP).

```
ret$ev[[1]]
```

```
[1] 202.5765
```

Calculating the EV across the entire opening portfolio yields the starting valuation figure:

```

df_test_start_sep <- df_test_start %>% fn_sep_to_estimated

ev_result_start <- fn_calc_ev(
  df_test_start_sep,
  model_xgboost_start, predfun_xgboost,
  verbose = 1) # Used later to construct the expected portfolio reflecting projected
  ↪ lapses.
sum(ev_result_start$ev)

```

3 Calculating EV Analysis of Movement

We perform an Analysis of Movement (AoM) following the methodology in the original paper, using the transition from year-end 2002 to year-end 2003 (evaluation year 2002) as our case study. Starting with the opening EV (`ev_result_start`), we step through each intermediate projection scenario.

First, treating the opening test portfolio as active contracts, we construct two closing portfolios: one decremented by expected lapses projected under beginning-of-period assumptions, and one decremented

by actual observed lapses.

```
df_test_start_end_pred <- df_test_start_sep %>%
  cbind(pred = ev_result_start$df_projected_data %>%
    filter(t == 0) %>% select(lapse_rate) %>% pull) %>%
  mutate(exposure = exposure * (1 - pred)) %>%
  mutate(exposure_amount = exposure_amount * (1 - pred)) %>%
  select(-pred) %>% fn_shift_year(year = 1)

df_test_start_end_act <- df_test_start_sep %>% fn_get_port_eopy
```

With all models and portfolios prepared, we evaluate the sequential scenarios. Scenario A computes EV using the expected portfolio under beginning-of-period assumptions to isolate the time-unwinding effect.

```
ev_result_a <- fn_calc_ev(
  df_test_start_end_pred,
  model_xgboost_start, predfun_xgboost,
  verbose = 0)
sum(ev_result_a$ev)
```

[1] 63730019

Next, Scenario B evaluates the actual portfolio under beginning-of-period assumptions. The difference between Scenario A and Scenario B defines the experience variance.

```
ev_result_b <- fn_calc_ev(
  df_test_start_end_act,
  model_xgboost_start, predfun_xgboost,
  verbose = 0)
sum(ev_result_b$ev)
```

[1] 64431748

Furthermore, Scenario C updates the prediction model to end-of-period assumptions. The difference between Scenario B and Scenario C isolates the operating assumption change.

```
ev_result_c <- fn_calc_ev(
  df_test_start_end_act,
  model_xgboost_end, predfun_xgboost,
  verbose = 0)
sum(ev_result_c$ev)
```

[1] 71543997

Finally, evaluating the closing portfolio at the end of the year yields Scenario $(y + 1)$ -End, completing the movement reconciliation.

```
ev_result_end <- fn_calc_ev(
  df_test_end %>% fn_sep_to_estimated,
  model_xgboost_end, predfun_xgboost,
```

```

verbose = 0)
sum(ev_result_end$ev)

```

[1] 145765359

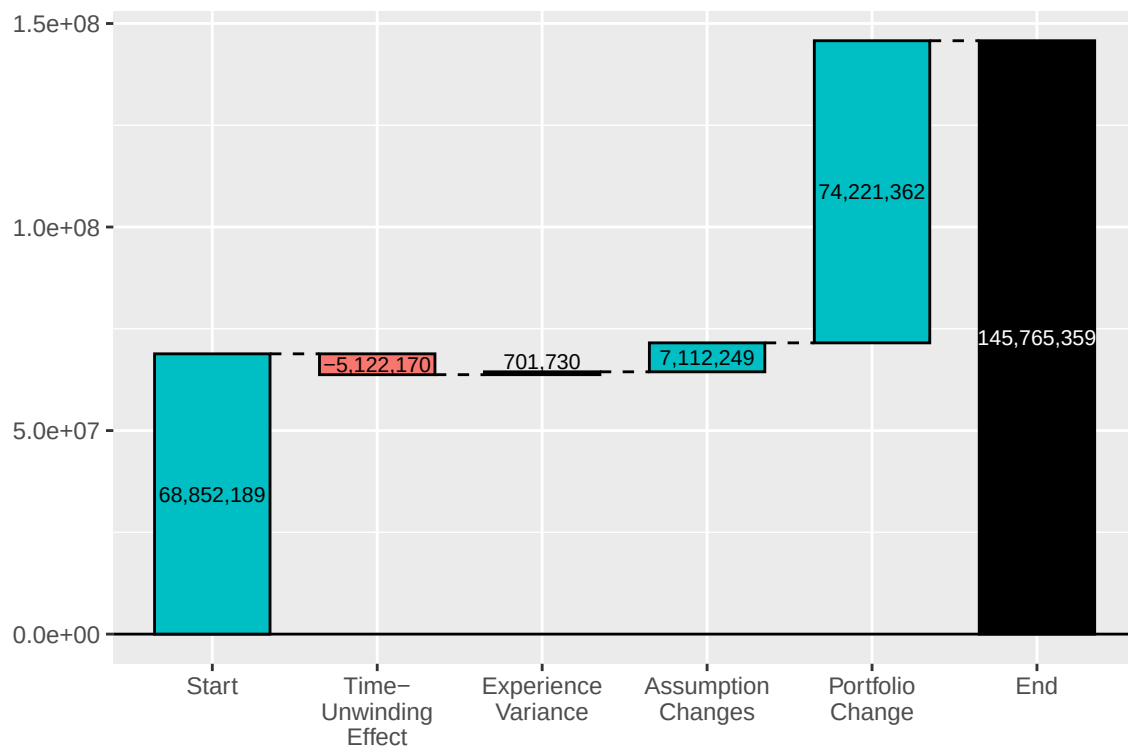
Computing the step-by-step variances and visualizing them via a waterfall chart replicates the figures reported in the original paper.

```

vec_values <- c(sum(ev_result_start$ev)
, sum(ev_result_a$ev) - sum(ev_result_start$ev)
, sum(ev_result_b$ev) - sum(ev_result_a$ev)
, sum(ev_result_c$ev) - sum(ev_result_b$ev)
, sum(ev_result_end$ev) - sum(ev_result_c$ev))

waterfall(
  labels = c(
    "Start", "Time-\nUnwinding\nEffect", "Experience\nVariance",
    "Assumption\nChanges", "Portfolio\nChange"),
  total_axis_text = "End",
  values = vec_values,
  rect_text_labels = scales::comma(vec_values, accuracy = 1),
  total_rect_text = scales::comma(sum(vec_values), accuracy = 1),
  calc_total = T
)

```



Acknowledgments

The authors gratefully acknowledge the Society of Actuaries (SOA) for providing the public dataset (SOA 10-Year Level Premium Term Lapse Study) utilized in this study. In addition, generative AI tools (such as Gemini and ChatGPT) were used to assist with English language editing and R code refinement, with all final content thoroughly reviewed and verified by the authors.

References

Kozuma, R., Asashiba, R., Matsumori, Y., & Iwasawa, H. (2026). *Enhancing GLM-based lapse rate models for EV calculation using interpretable machine learning.*